

Índice:

- Prácticas 1 - 10
- Prácticas 11 - 20
- Prácticas 21 - 30
- Prácticas 31 - 37

Crear el script como se ha comentado sustituyendo “alumno” por nuestro nombre completo. Tomar captura de pantalla del resultado.

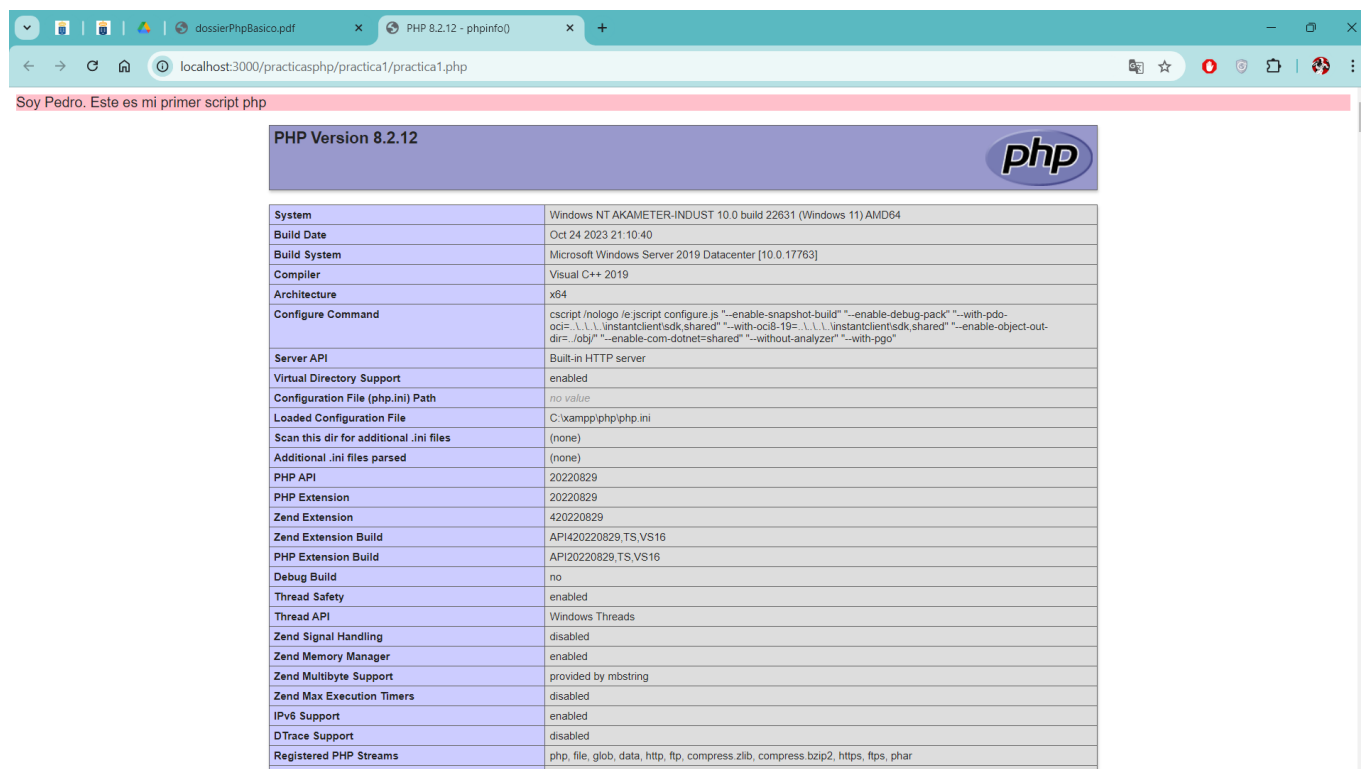
```
<?php

echo "<p style='background: pink'>Soy Pedro. Este es mi primer script php";

echo "</p>";

phpinfo();

?>
```



Practica 2

Crear el script anterior. Modificarlo para sumar a `un_str` el valor de `un_int` y mostrarlo en pantalla ¿ qué ocurre ? .

Sumar `un_str` con `un_str2` ¿ qué ocurre ? ¿ se puede concatenar una cadena con comillas simples con una con comillas dobles ?

```
<?php

$un_bool = TRUE; //booleano

$un_str = "foo"; //cadena de caracteres con doble comilla

$un_str2 = 'foo'; //cadena de caracteres con comilla simple

$un_int = 12; //número entero


echo gettype($un_bool); // imprime: boolean
echo gettype($un_str); // imprime: string


// Si este valor es un entero, incrementarlo en cuatro

if (is_int($un_int)) {
    $un_int += 4;
}


// Si $un_bool es una cadena, imprimirla
// (no imprime nada)
if (is_string($un_bool)) {
    echo "Cadena: $un_bool";
}


echo $un_str + $un_int;

echo $un_str + $un_str2;
?>
```

Al sumar `un_int` a `un_str` da error, por que el primero es un entero y el segundo un string.

Al sumar `un_str` con `un_str2` da error también, por que no se pueden concatenar dos string con diferentes tipos de comilla.

Practica 3

Realizar el código anterior y tomar captura de pantalla del resultado. ¿qué es lo que ha ocurrido ?.

Poner código html antes de la declaración de `strict_types` y probar de nuevo ¿ qué ocurre ahora ?

```
<?php
```

```
declare( strict_types=1);
//La declaración del tipado va siempre al principio, para restringir
los tipos
?>

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <meta name="author" content="Pedro Martin Escuela" />
    <title>Practica 3</title>
    <script src="practica3script.php"></script>
  </head>

  <body>
    <?php
      function sum($a, $b) : int{
        return $a + $b;
      }

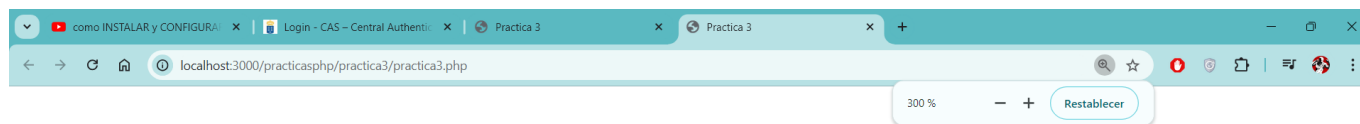
      echo "<p> la suma de uno más dos es: ";
      $resultado = sum(1,2);
      print sum(1,2);
      echo "</p>"

      //Práctica 3
      //Realizar el código anterior y tomar captura de pantalla del
      resultado. ¿qué es lo que ha ocurrido ?.
      //Poner código html antes de la declaración de strict_types y
      probar de nuevo ¿ qué ocurre ahora ?

      //Primero, el código funcionaba correctamente, al abrirlo en el
      php server, se podía ver la sentencia
      //y el resultado sumado (captura 1). Luego de cambiarlo y poner
      la declaración dentro del html, despues
      //de declarar el html, se marca un error que nos especifica que
      la declaración debe ser declarado lo
      //primero de todo (captura 2).

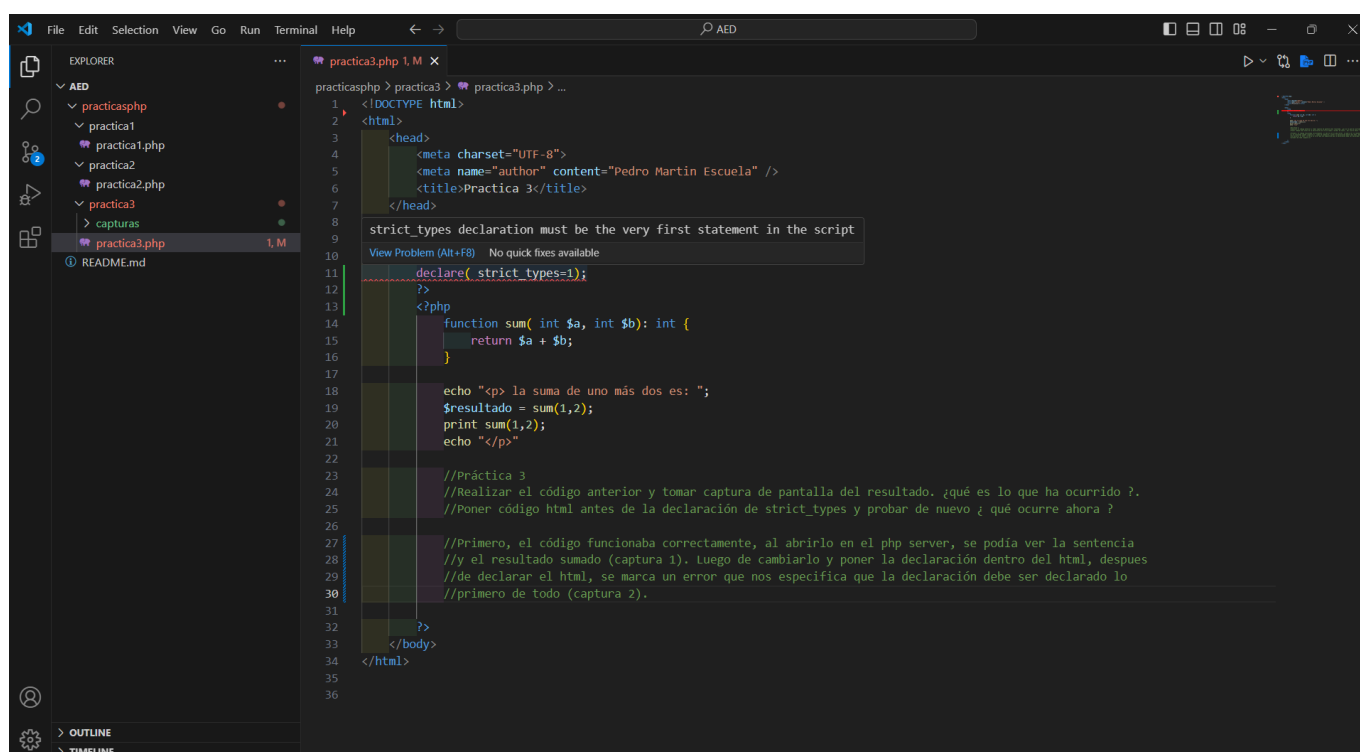
    ?>
  </body>
</html>
```

Primero, el código funcionaba correctamente, al abrirlo en el php server, se podía ver la sentencia y el resultado sumado (captura 1).



la suma de uno más dos es: 3

Luego de cambiarlo y poner la declaración dentro del html, después de declarar el html, se marca un error que nos especifica que la declaración debe ser declarada lo primero de todo (captura 2).



Practica 4

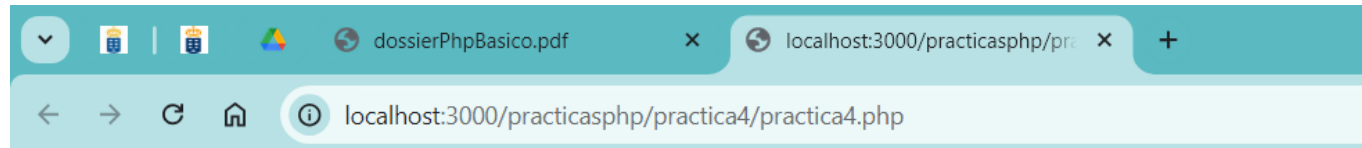
Tomar captura de pantalla de: (y explicar lo ocurrido)

- probar el código anterior ¿ da error ? Por qué ?
- quitar el comentario a: return \$a; ¿ da error ahora ? por qué ?
- quitar comentario a: print fun("e",3); ¿ da error ?

```
<?php
    declare( strict_types=1);
?>
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title></title>
    </head>
    <body>
        <?php
            function fun( int $a, int $b): int {
                $a = "o";
                //return $a;
                return $b ;
            }
            print fun(1,2);
            //print fun("e",3);
            echo "</p>"

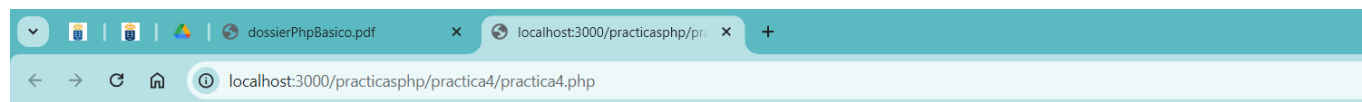
        ?>
    </body>
</html>
```

No da error, por que está devolviendo la b, que está restringido a entero y está devolviendo un entero.



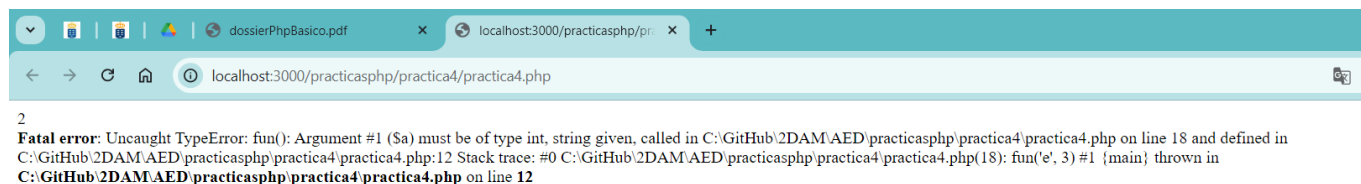
2

Da error, por que está restringido a enteros y el a ha sido refactorizado a string.



Fatal error: Uncaught TypeError: fun(): Return value must be of type int, string returned in C:\GitHub\2DAM\AED\practicashp\practica4\practica4.php:14 Stack trace: #0 C:\GitHub\2DAM\AED\practicashp\practica4\practica4.php(17): fun('o', 2) #1 {main} thrown in C:\GitHub\2DAM\AED\practicashp\practica4\practica4.php on line 14

Da error, por que se le está introduciendo un string y sigue devolviendo un string, cuando los tipos están restringidos a entero.



Practica 5

Practica 5.1: Probar el código anterior. Probar ahora con números ¿ también funcionan las referencias ?

```
<?php
    $foo = 'Bob'; // Asigna el valor 'Bob' a $foo
    $bar = &$foo; // Referencia $foo vía $bar.
    $bar = "Mi nombre es $bar"; // Modifica $bar...
    echo $foo; // $foo también se modifica.
    echo $bar;

    $numero1 = 3;
    $numero2 = &$numero1;
    $numero2 = 5;

    echo $numero2;
    echo $numero1;

?>
```

Si, con numeros también funcionan las referencias.

Practica 6

Hacer un script php que haga echo de `_SERVER` y de `_SERVER [PHP_SELF]` tomar captura de pantalla de los resultados.

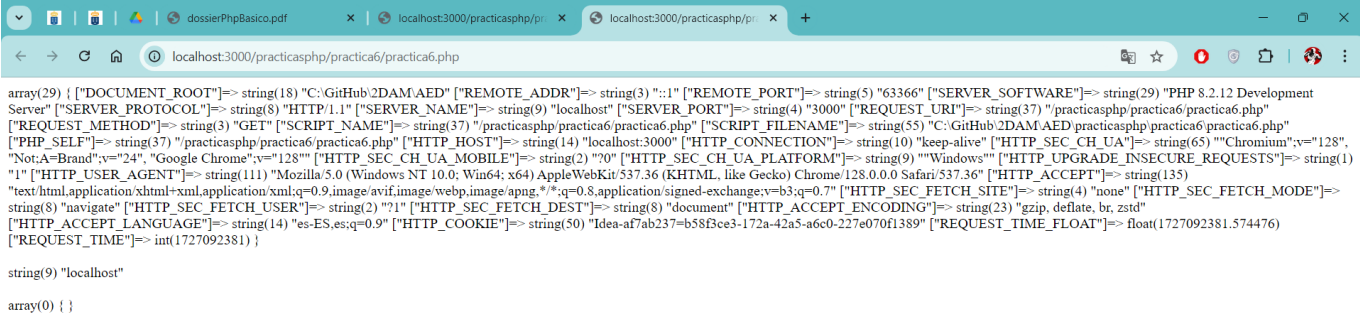
```
<?php

//var_dump($_SERVER); //Info del servidor, puerto, localhost, desde donde
se lanza el archivo, etc. Info del servidor

//var_dump($_SERVER["SERVER_NAME"]); //Enseña exactamente el punto del
array que estás indicando

var_dump($_REQUEST); //La petición (GET, POST, DELETE, ...)

?>
```



Practica 7

Visualizar lo anterior ¿ se encuentran diferencias entre null y unset() ? Tomar captura de pantalla

```
<?php

$variable = null;

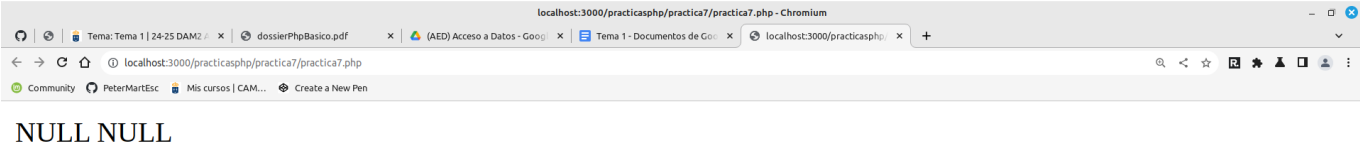
var_dump($variable);

unset($variable);

var_dump($variable);

?>
```

No se encuentras diferencias



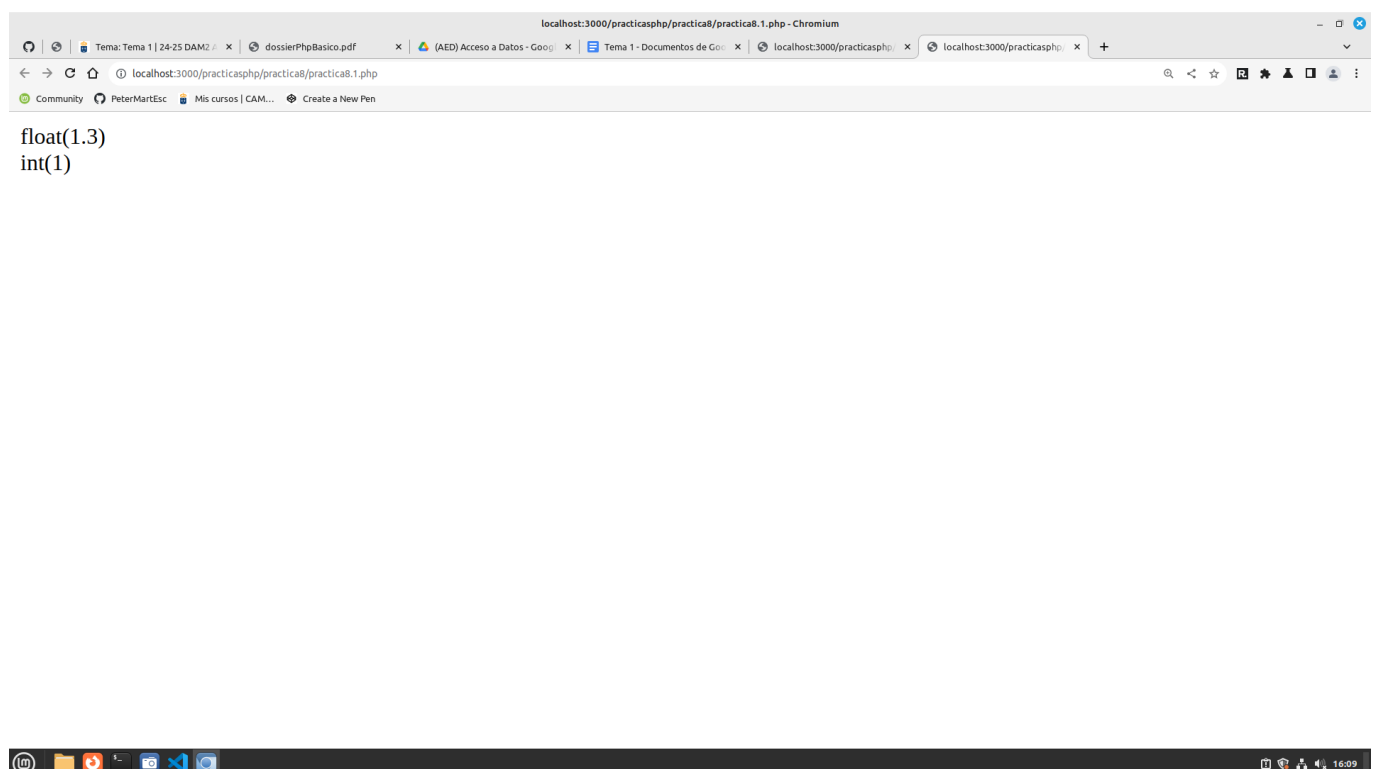
Practica 8

Ejecutar el script anterior ¿ hay alguna diferencia antes y después del cast ?

Tomar captura de pantalla

```
<?php
    $unavar = 1.3;
    var_dump($unavar);
    echo "<br>";
    $unavar = (int) $unavar;
    var_dump($unavar);
?>
```

Si hay diferencias, se cambia el tipo de variable de float a int y además se trunca el numero de 1.3 a 1



Practica 8.2

¿ qué ámbito tienen las constantes ? ¿ realmente no se puede poner varios valores en un constante ?

- Probar fuera de una función a crear constante: `const PULGADA = 2.53;`
- Ahora tratar de establecerla de nuevo mediante:
`PULGADA = 7; const PULGADA = 8; $PULGADA = 9;`
- Hacer echo en cada caso.
- Crear la constante en ámbito global (fuera de función) ¿ se puede acceder dentro de una función ? ¿ se puede establecer: `const PULGADA = 10` dentro de una función ?
- Tomar capturas de pantalla en cada caso y explicar lo que ha ocurrido

```
<?php

//const PULGADA = 2.53;
//PULGADA = 7;

//echo $PULGADA;

const PULGADA = 8;
$PULGADA = 9;

echo $PULGADA;

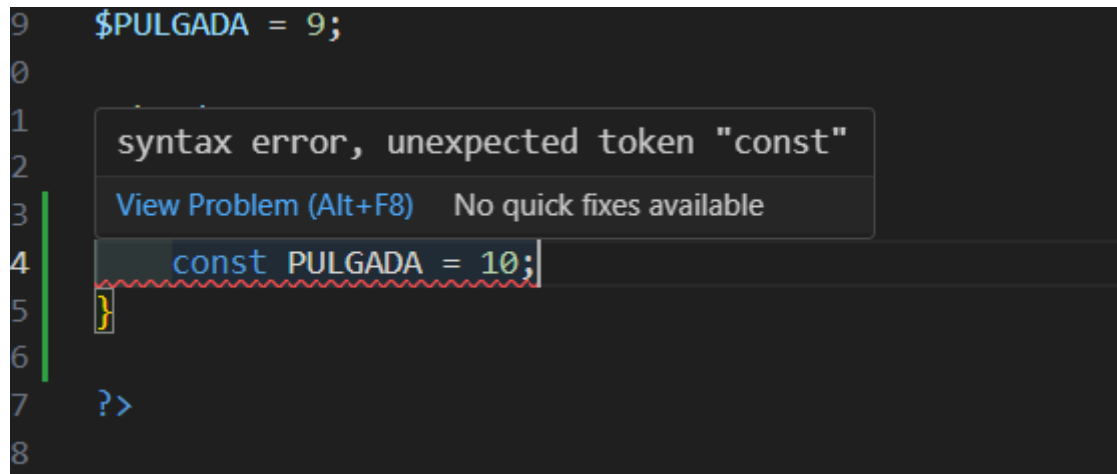
function comprobarConstante(){
    echo $PULGADA;
}

comprobarConstante();

?>
```

Las constantes tienen ámbito de bloque, tienen que ser definidas una vez declaradas y no pueden cambiar de valor.

La variable no se puede acceder desde dentro de la función, al llamarla, dice que no está definida.



Practica 9

Crear un script que muestre las potencias del número 2 desde 2^1 hasta 2^9 hacer uso del operador: `**` Ir concatenando las salidas en pantalla de las potencias en una string mediante el operador de concatenación y asignación: `.=`

```
<?php

$base = 2;
$potenciasConcat = "";

for ($i = 1; $i <= 9; $i++){
```

```

    $potenciasConcat .= " " . $base ** $i;
    //echo $potenciasConcat;
}

echo $potenciasConcat;

?>

```

Practica 10

Crear un programa en php que obtenga la descomposición de un número que esté almacenado en la variable: \$numero Por ejemplo: numero = 3102 Se pretende que se utilicen en el programa los operadores: ., **, Para el ejemplo anterior se debe mostrar en pantalla: 2 * 1 + 0 * 10 + 1 * 100 + 3 * 1000

```

<?php

//declare( strict_types = 1 );

$numero = "123";
$resultado = "";
$multiplicador = 1;

for ($i = 0; $i < strlen($numeroStr); $i++){

    if($i == (strlen($numeroStr)-1)){
        $numeroSuelto = $num[$i] . "*" . $multiplicador;
    } else {
        $numeroSuelto = $num[$i] . "*" . $multiplicador."+";
        $multiplicador *= 10;
    }

}

echo $resultado;

?>

```

Practica 11

Ejecutar el script y tomar captura de pantalla

```

<?php
$var = "";

if(empty($var)){ // true because "" is considered empty
    echo '<br>empty($var) para $var="" ';
}

```

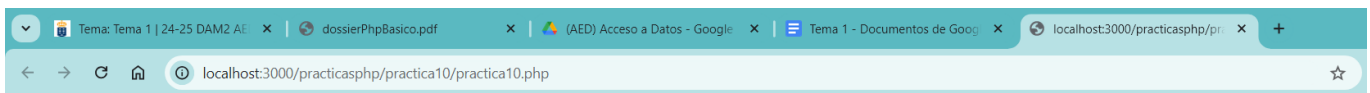
```

}else{
    echo '<br>!empty($var) para $var="" ' ;
}
if(isset($var)){ //true because var is set
    echo '<br>isset($var) para $var="" ' ;
}else{
    echo '<br> !isset($var) para $var="" ' ;
}

if(empty($otherVar)){ //true because $otherVar is null
    echo '<br>empty($otherVar) para $otherVar que no se ha establecido ' ;
} else {
    echo '<br> !empty($otherVar) para $otherVar que no se ha establecido ' ;
}
if(isset($otherVar)){ //false because $otherVar is not set
    echo '<br>isset($otherVar) para $otherVar que no se ha establecido ' ;
} else {
    echo '<br> !isset($otherVar) para $otherVar que no se ha establecido ' ;
}

?>

```



empty(\$var) para \$var=""
 isset(\$var) para \$var=""
 empty(\$otherVar) para \$otherVar que no se ha establecido
 !isset(\$otherVar) para \$otherVar que no se ha establecido

Practica 12

Probar el script anterior y observar que ocurre. ¿ qué mensaje de error se observa ?

```

<?php
    $array = array('uno' => 1, 'dos' => 2, 'tres' => 40, 'cuatro' => 55);
    $cadena = "La posición 'tres' contiene el dato {$array['tres']}";
    echo $cadena;

?>

```

Se muestra un error de sintaxis, dice que se ha contrado un elemento inesperado en el string

Practica 13

Probar el script anterior y observar que ocurre. Probar ahora con llaves: \${\$variable} ¿ hay diferencia ?

```
<?php
    $variable = 'dato';
    $dato = 5;
    echo ${$variable}.'<br>';
?>
```

No parece haber diferencia, se muestra el 5 también. En caso de colocarle los corchetes a todo, si dice que no encuentra la variable \$dato

Practica 14

Toma el código anterior e introduce una expresión “variable de variables” que permita definir las variables: \$dato0, \$dato1, ..., \$dato9 Cada una de ellas con el valor correspondiente: 0, 1,...,9

```
<?php

    for($i=0;$i<10;$i++){
        $variable = "dato";
        ${$variable.$i} = $i;
        //echo ${"dato".$i};
    }

    echo "<br> $dato3 ";
    echo "<br> $dato8 ";
?>
```

Practica 15

Ejecutar el script anterior

- ¿ se muestran las posiciones anteriores a la 2 ?
- ¿ y entre la 2 y la 7 ?
- Realizar el mismo script pero en lugar de crear el array mediante los corchetes: \$array = [] hacerlo con la función array()
- ¿ hay diferencias en la salida en pantalla ?
- Ejecutar var_dump(\$array) después de cada asignación de un valor al array.
- Tomar captura de pantalla de los resultados

```
<?php
    $array = [];
    //$array();
    $array[2]="mensaje";
```

```
var_dump($array);  
$array[7]="lalala!";  
var_dump($array);  
$array[]="yepa yepa!!";  
var_dump($array);  
?>
```

Con los corchetes, no se muestran las posiciones anteriores al 2 ni entre el 2 y el 7. Hay diferencia, dice que no están definidos los valores de cada array

Practica 15.2

crear un array asociativo dejando sin poner en algunas ocasiones la parte de la clave dejando únicamente el valor (al estilo de si fuera un array no asociativo) hacer un var_dump() y recorrerlo con un for (no con un foreach) ¿ muestra algún valor ? ¿ genera error ?

```
<?php  
  
$array = array(  
  
    "espada" => "arma",  
    "100" => "durabilidad",  
    "usuario",  
    "cofres" => "loot"  
  
);  
  
var_dump($array);  
  
for($i = 0; $i < count($array);$i++){  
    echo $array[$i] . "\n";  
}  
  
?>
```

Muestra solo los valores cuyas claves no hemos sustituido por cadenas de texto, en este caso, muestra "usuario".

Practica 16

Ejecutar el script anterior. ¿ Tenemos que usar los nombres de variables \$key y \$val ? Sustituir por otros nombres de variables y ver si hay algún problema

```
<?php  
$array = array('perro', 'gato', 'avestruz');  
  
foreach ($array as $key => $val) {  
    print "<br>array[ $key ] = $val";  
}
```

```
}  
?>
```

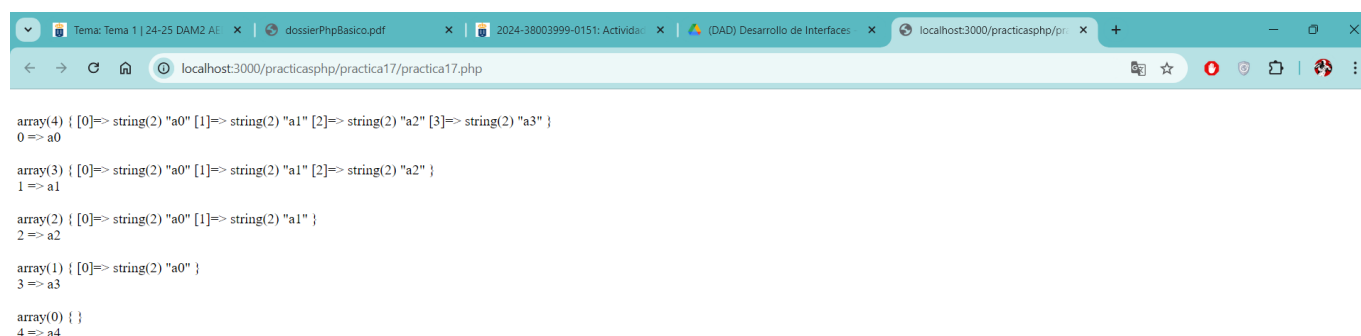
No, no es necesario utilizar estos nombres, aun que si es recomendado para que sean representativos, pero al utilizar otros nombres en las variables, sigue funcionando, supongo que por que se le asigna su función en el propio for each?

Practica 17

Ejecutar el script anterior. En Java eliminar elementos de un array en un foreach implica un error ¿ también en php ? Tomar captura de pantalla del resultado

```
<?php  
    $array = [];  
  
    for($i=0;$i<5;$i++){  
        $array[] = "a" . $i;  
    }  
  
    $j=count($array);  
  
    foreach( $array as $key => $val){  
        $j--;  
        unset($array[$j]);  
        echo "<br>";  
        var_dump($array);  
        echo "<br> $key => $val ";  
        echo "<br>";  
    }  
?>
```

No parece haber un error



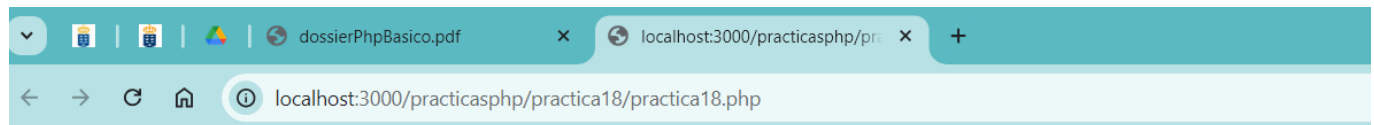
```
array(4) { [0]=> string(2) "a0" [1]=> string(2) "a1" [2]=> string(2) "a2" [3]=> string(2) "a3" }  
0 => a0  
  
array(3) { [0]=> string(2) "a0" [1]=> string(2) "a1" [2]=> string(2) "a2" }  
1 => a1  
  
array(2) { [0]=> string(2) "a0" [1]=> string(2) "a1" }  
2 => a2  
  
array(1) { [0]=> string(2) "a0" }  
3 => a3  
  
array(0) { }  
4 => a4
```

Practica 18

Ejecutar el script anterior. Modificar los echo para que se sepa cuando llamamos a \$array (recordar que con comillas simples no interpreta) Tomar captura de pantalla

```
<?php
    $array = ["a", "a", "a", "a", "a"];

    $j=count($array);
    foreach( $array as $key => &$val){
        $j--;
        $array[$j] .= $j;
        echo "<br>";
        var_dump($array);
        echo "<br> $key => $val";
        echo "<br> $key => $array[$key]";
        echo "<br>";
    }
?>
```



```
array(5) { [0]=> &string(1) "a" [1]=> string(1) "a" [2]=> string(1) "a" [3]=> string(1) "a" [4]=> string(2) "a4" }
0 => a
0 => a

array(5) { [0]=> string(1) "a" [1]=> &string(1) "a" [2]=> string(1) "a" [3]=> string(2) "a3" [4]=> string(2) "a4" }
1 => a
1 => a

array(5) { [0]=> string(1) "a" [1]=> string(1) "a" [2]=> &string(2) "a2" [3]=> string(2) "a3" [4]=> string(2) "a4" }
2 => a2
2 => a2

array(5) { [0]=> string(1) "a" [1]=> string(2) "a1" [2]=> string(2) "a2" [3]=> &string(2) "a3" [4]=> string(2) "a4" }
3 => a3
3 => a3

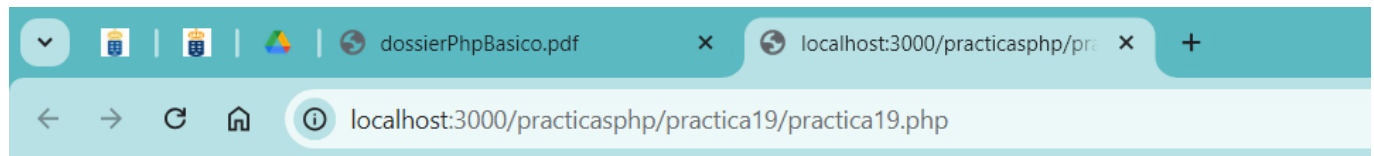
array(5) { [0]=> string(2) "a0" [1]=> string(2) "a1" [2]=> string(2) "a2" [3]=> string(2) "a3" [4]=> &string(2) "a4" }
4 => a4
4 => a4
```

Practica 19

Ejecutar el script anterior. Tomar captura de pantalla del resultado

```
<?php
    $arr = array(1, 2, 3, 4);
    foreach ($arr as &$val) {
        $val = $val * 2;
    }
}
```

```
        foreach ($arr as $key => $val) {  
            echo "{$key} => {$val} <br>";  
            print_r($arr);  
            echo "<br><br>";  
        }  
    ?>
```

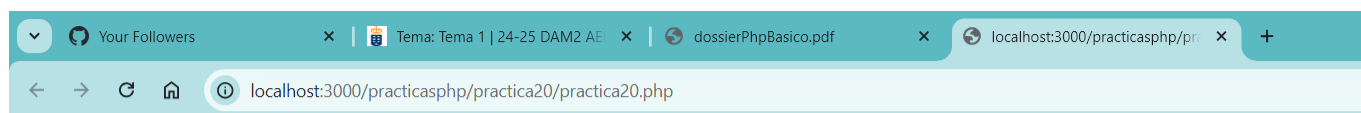


```
0 => 2  
Array ( [0] => 2 [1] => 4 [2] => 6 [3] => 2 )  
  
1 => 4  
Array ( [0] => 2 [1] => 4 [2] => 6 [3] => 4 )  
  
2 => 6  
Array ( [0] => 2 [1] => 4 [2] => 6 [3] => 6 )  
  
3 => 6  
Array ( [0] => 2 [1] => 4 [2] => 6 [3] => 6 )
```

Practica 20

Ejecutar el script anterior. ¿ qué valor devuelve ?Tomar captura de pantalla

```
<?php  
    $arr= ["1", "2", "3", "4"];  
    $va = array_pop($arr);  
    echo "el array ahora queda: <br>";  
    print_r($arr);  
    echo "<br>el valor extraido es: " . $va;  
    ?>
```



el array ahora queda:
Array ([0] => 1 [1] => 2 [2] => 3)
el valor extraido es: 4

Practica 21

Crear un script que por medio de un bucle for que vaya de 1 a 10 agregue esos números en un array.

En cada iteración mostrar el contenido del array. Después en un bucle for de 1 a 5 ir ejecutando sentencias `array_pop()` y mostrar como queda el array en cada iteración

```
<?php

$array = [];

for($i = 1; $i<=10;$i++){
    $array[$i-1] = $i;
    echo "El array queda: <br>";
    print_r($array);
    echo "<br>";
}

echo "<br><br>";

for($i = 1; $i<=5;$i++){
    array_pop($array);
    echo "El array después del 'pop' queda: <br>";
    print_r($array);
    echo "<br>";
}

?>
```

Practica 22

Crear un script que por medio de un bucle for que vaya de 1 a 10 agregue esos números en un array mediante `array_unshift()`. En cada iteración mostrar el contenido del array. Después en un bucle for de 1 a 5 ir ejecutando sentencias `array_shift()` y mostrar como queda el array en cada iteración

```
<?php

$array = [];

for($i = 1; $i<=10;$i++){
    array_unshift($array, $i);
    echo "El array queda: <br>";
    print_r($array);
    echo "<br>";
}

echo "<br><br>";

for($i = 1; $i<=5;$i++){
    array_shift($array);
    echo "El array después del 'shift' queda: <br>";
    print_r($array);
    echo "<br>";
}

?>
```

Practica 23

Haz una página PHP que utilice foreach para mostrar todos los valores del array \$_SERVER en una tabla con dos columnas. La primera columna debe contener el nombre de la variable, y la segunda su valor

PHP:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <meta name="author" content="Pedro Martín Escuela">
    <title>Tabla SERVER php</title>
</head>
<body>
    <div class="contenedorTabla">
        <?php
            //print_r($_SERVER);
            echo "<table>";
            foreach ($_SERVER as $key => $val) {

                echo "<tr><td>{$key}</td><td>{$val}</td></tr>";

            }
            echo "</table>";
        </?php>
    </div>
</body>
</html>
```

```
        ?>
    </div>
</body>
</html>
```

CSS:

```
table, td, tr{
    border: solid 1px;
}
```

Practica 24

Ejecutar los códigos de `in_array()`, `array_search()`, `array_values()` tomar captura del resultado de la ejecución

```
<!-- in_array-->

<?php
    $a = array('1.10', 12.4, 1.13);
    if (in_array('12.4', $a, true)) {
        echo "Se encontró '12.4' con comprobación estricta\n";
    }
    if (in_array(1.13, $a, true)) {
        echo "Se encontró 1.13 con comprobación estricta\n";
    }

    echo "<br>";
?>

<!-- array_search -->

<?php
    $array = array(0 => 'azul', 1 => 'rojo', 2 => 'verde', 3 => 'rojo');

    $clave = array_search('verde', $array);

    echo $clave . "<br>";

    $clave = array_search('marrón', $array);

    if( $clave === FALSE)
        //Cuando no se localiza, devuelve un FALSE
        echo "no se ha localizado el valor";
    else
        echo $clave;

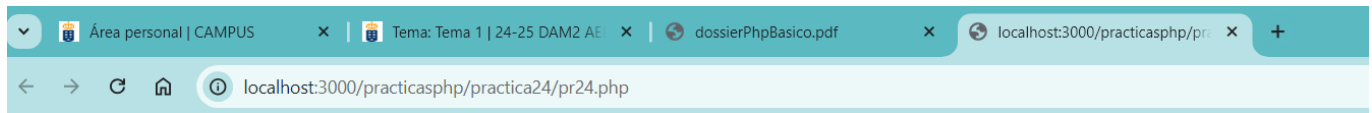
    echo "<br>";
?>
```

```
<!-- array_values -->

<?php
    $array = array('azul', 'rojo', 'verde', 'amarillo', "blanco");
    unset($array[2]);
    unset($array[3]);

    print_r($array);

    //ahora usando array_values "limpiamos"      (A que se refiere con limpiar?
    los huecos en blanco?)
    $array = array_values($array);
    echo "<br>";
    print_r($array);
?>
```



Se encontró 1.13 con comprobación estricta
2
no se ha localizado el valor
Array ([0] => azul [1] => rojo [4] => blanco)
Array ([0] => azul [1] => rojo [2] => blanco)

Practica 25

Rellenar un array con 10 números aleatorios entre 20 y 25 (hacer uso de: `rand ( int min , int max ) : int`) y luego hacer uso de `array_search()` para localizar el valor: "22" Se debe mostrar en pantalla el array completo y el valor devuelto por `array_search()`

```
<?php

$min = 20;
$max = 25;

$array = [];

for($i = 0 ; $i < 10 ; $i++ ){
    $array[$i] = generarNumAleatorio($min, $max);
}

function generarNumAleatorio(int $min , int $max) : int{
    return rand($min, $max);
}

print_r($array);
```

```
echo "<br><br>";  
$key = array_search(22, $array);  
echo "El valor devuelto por el array_search es $key"  
  
?>
```

Practica 26

Variar el ejemplo anterior para que se haga uso del operador nave espacial: <=>

```
<?php  
  
function cmp($a, $b){  
    //if ($a == $b) {  
        // return 0;  
    //}  
  
    return ($a <=> $b);  
}  
  
$a = array(3, 2, 5, 6, 1);  
  
usort($a, "cmp");  
  
foreach ($a as $valor) {  
    echo " $valor, ";  
}  
  
?>
```

Practica 27

Crear un array con los valores: [7,2,8,1,9,4] Hacer búsqueda con array_search() de: 4 Ordenar el array con usort y repetir la búsqueda de: 4 Mostrar los array antes y después de ordenación así como lo que devuelve array_search()

```
<?php  
  
function cmp($a, $b){  
    //if ($a == $b) {  
        // return 0;  
    //}  
  
    return ($a <=> $b);  
}  
  
$a = array(7, 2, 8, 1, 9, 4);  
  
usort($a, "cmp");  
  
foreach ($a as $valor) {  
    echo " $valor, ";  
}
```

```
$array = array(7,2,8,1,9,4);

$key = array_search(4, $array);
echo "El elemento 4 antes del usort está en la posición: $key";
echo "<br>";

print_r($array);
echo "<br><br>";

usort($array, "cmp");

$keyAFT = array_search(4, $array);
echo "El elemento 4 después del usort está en la posición: $keyAFT";
echo "<br>";
print_r($array);

//foreach ($array as $valor) {
    //echo " $valor, ";
//}

?>
```

Practica 28

Modifica el código anterior y quita el valor por defecto del parámetro \$print. Ejecuta el programa y toma captura de pantalla de los mensajes del IDE y responde: ¿ se obtiene resultado o se detiene el programa ?

```
<?php

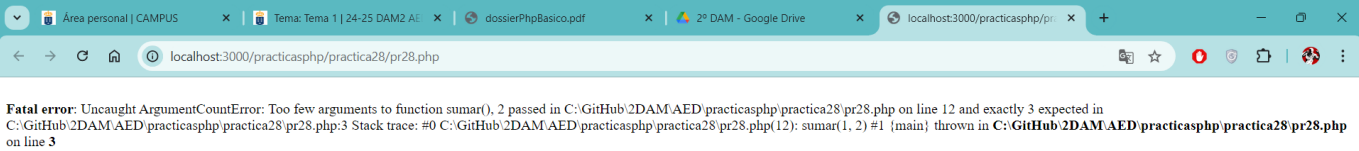
function sumar($a, $b, $print = false): float{
    $suma = $a + $b;
    if ($print) {
        echo "resultado suma: $suma <br>";
    }
    return $suma;
}

$sum1=sumar(1,2);
$sum2=sumar(4,5,true);

echo "las operaciones para sum1 y sum2 dan: $sum1 , $sum2";

?>
```

Se detiene el programa, no se obtiene resultado.

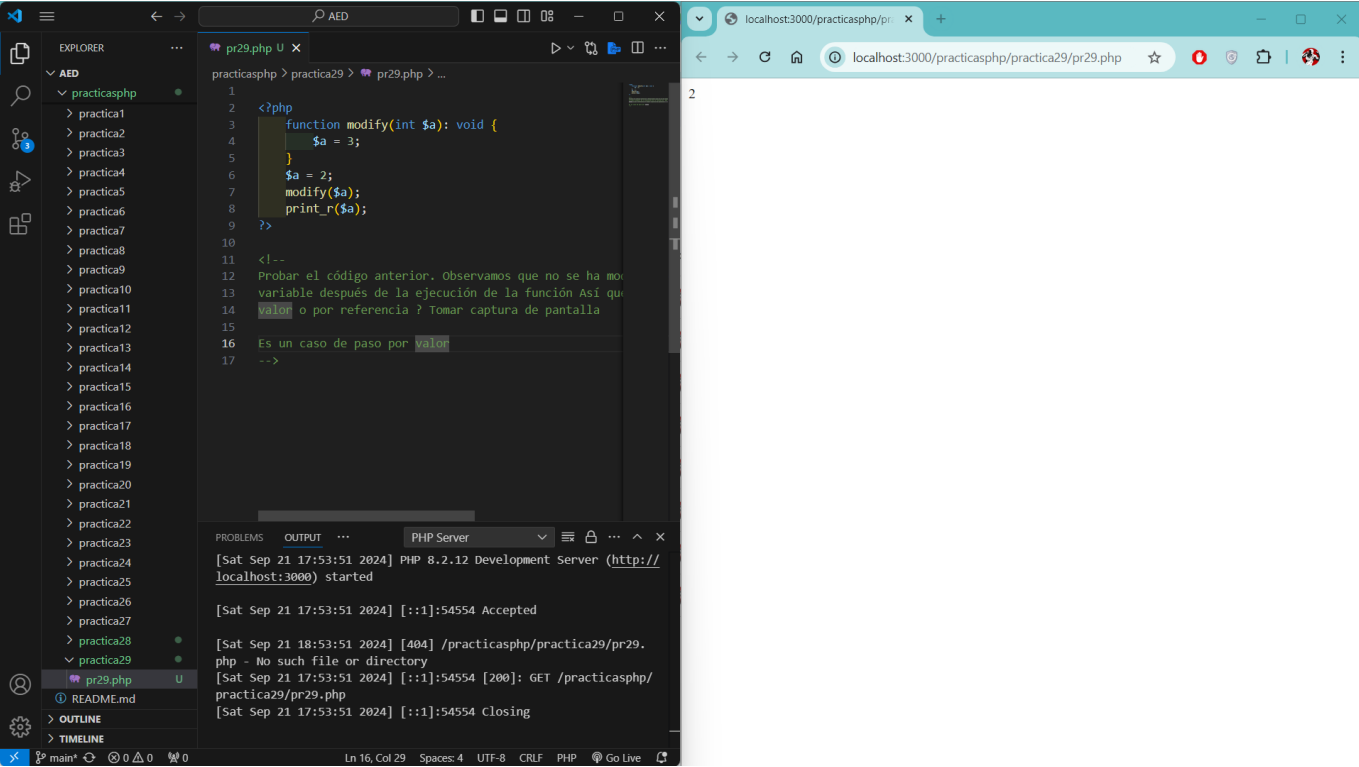


Practica 29

Probar el código anterior. Observamos que no se ha modificado el valor de la variable después de la ejecución de la función Así que ¿ estamos en un caso de paso por valor o por referencia ? Tomar captura de pantalla

```
<?php
function modify(int $a): void {
    $a = 3;
}
$a = 2;
modify($a);
print_r($a);
?>
```

Es un caso de paso por valor



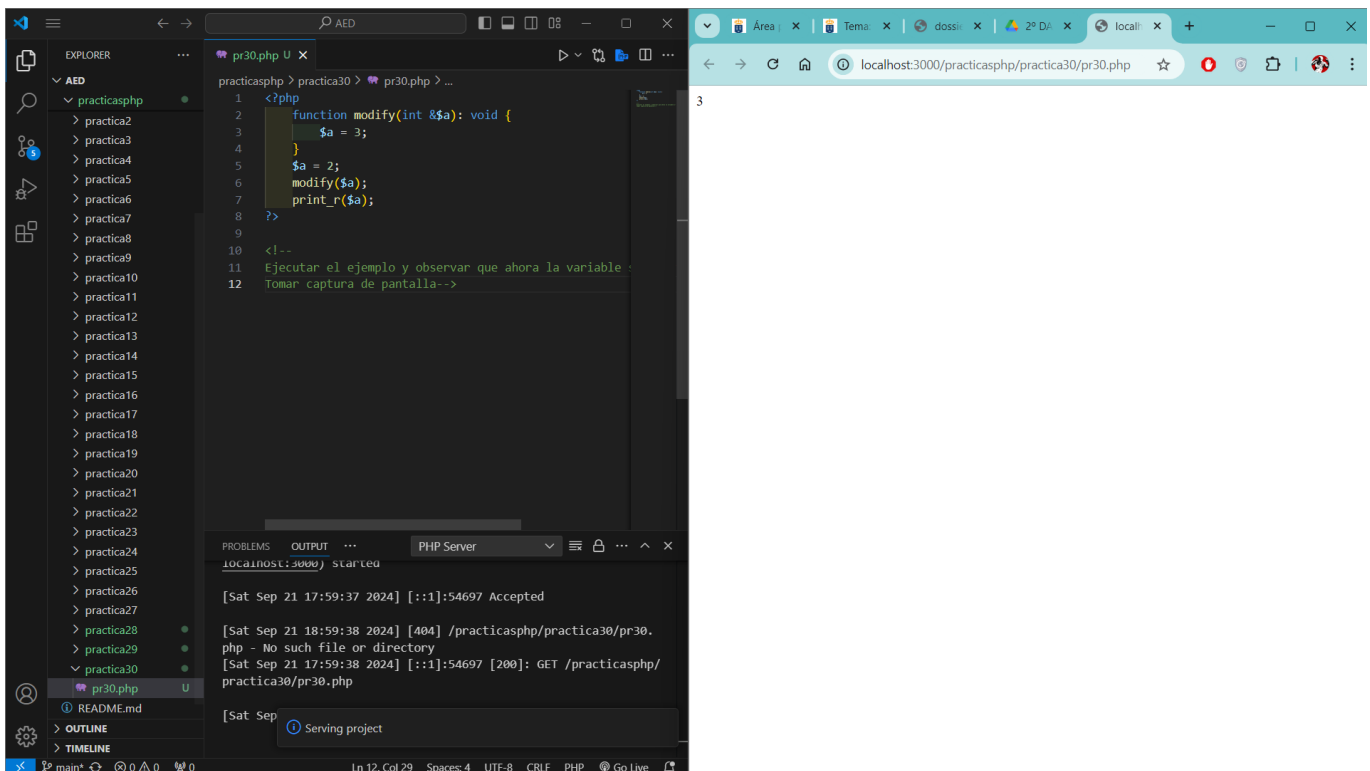
Practica 30

Ejecutar el ejemplo y observar que ahora la variable sí se ve modificada. Tomar captura de pantalla



```
<?php
function modify(int &$a): void {
    $a = 3;
}
$a = 2;
modify($a);
print_r($a);

?>
```



Practica 31

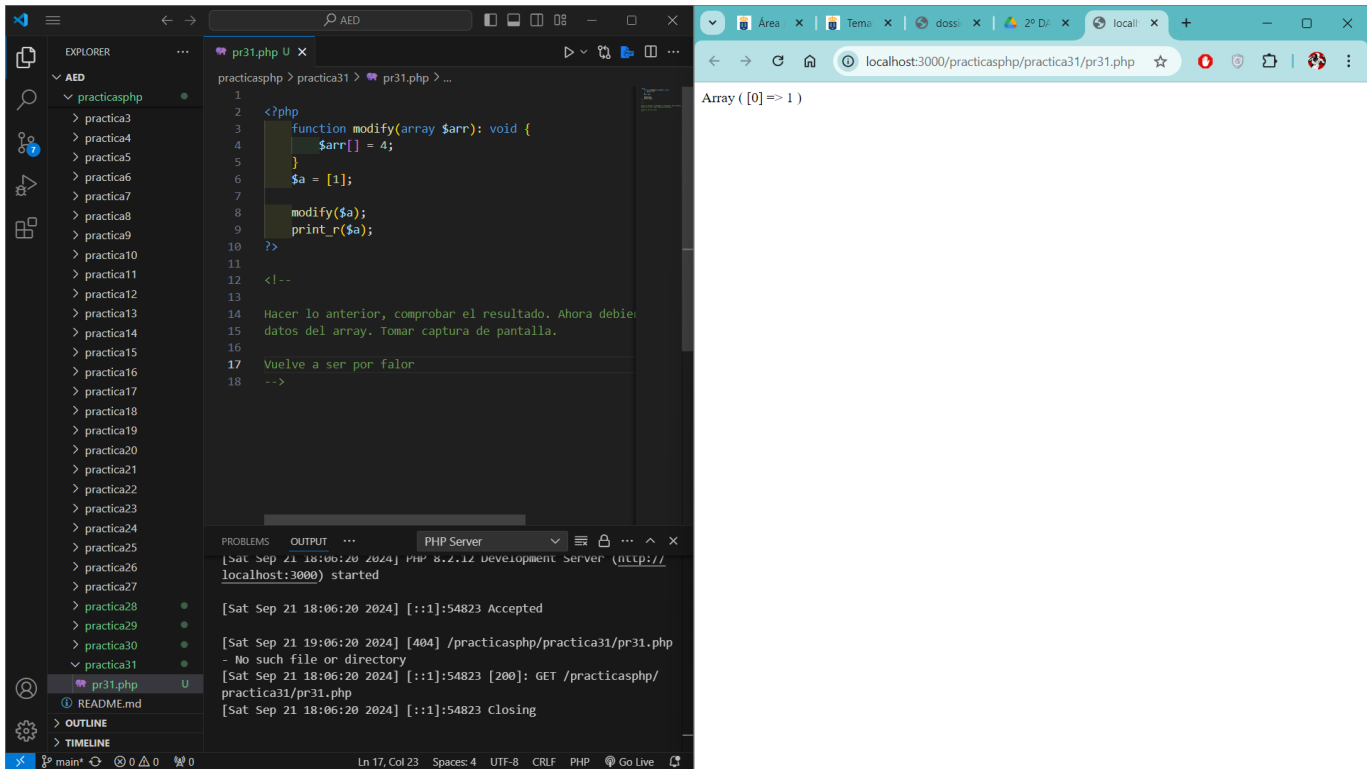
Hacer lo anterior, comprobar el resultado. Ahora debiera mostrar todos los datos del array. Tomar captura de pantalla.

```
<?php
function modify(array $arr): void {
    $arr[] = 4;
}
$a = [1];

modify($a);
print_r($a);

?>
```

Vuelve a ser por valor



Practica 32

Hacer lo anterior, pero usando require en lugar de include. Para que se note la diferencia la llamada de require debiera ser a un nombre de fichero incorrecto. Por ejemplo haremos que llame a: vars1.php cuando como sabemos el fichero es vars.php Tomar captura de pantalla

test.php :

```
<?php

require 'vars.php';
echo "Una $fruta $color"; // Una
echo "Una $fruta $color"; // Una manzana verde
?>
```

vars.php :

```
<?php
    $color = 'verde';
    $fruta = 'manzana';
?>
```

El include debe ir al principio del codigo o no lo detectará la primera linea

Practica 33

Hacer lo anterior, pero se debe comprobar la diferencia de pasar el texto con urlencode y sin urlencode. Así que se propone poner dos parámetros: prueba y prueba2 uno de ellos con urlencode y el otro sin él pasando en ambos casos el mismo texto en el value. Tomar captura de pantalla de lo obtenido

```
<?php

$texto = "Pasando datos diría.. que hay que usar urlencode";

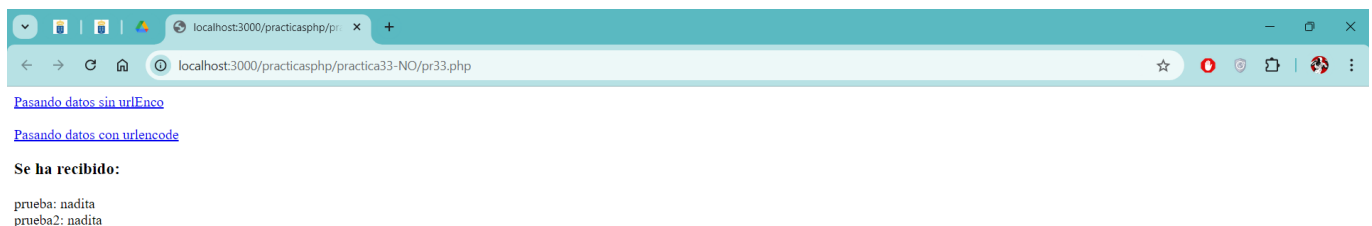
// Codificamos el texto con urlencode
$conUrlEncode = urlencode($texto);

// Creamos un enlace con los dos parámetros: prueba (sin urlencode) y
prueba2 (con urlencode)
echo "<a href='pr33.php?prueba={$texto}'>Pasando datos sin urlEnco</a>";
echo "<br><br>";
echo "<a href='pr33.php?prueba2={$conUrlEncode}'>Pasando datos con
urlencode</a>";

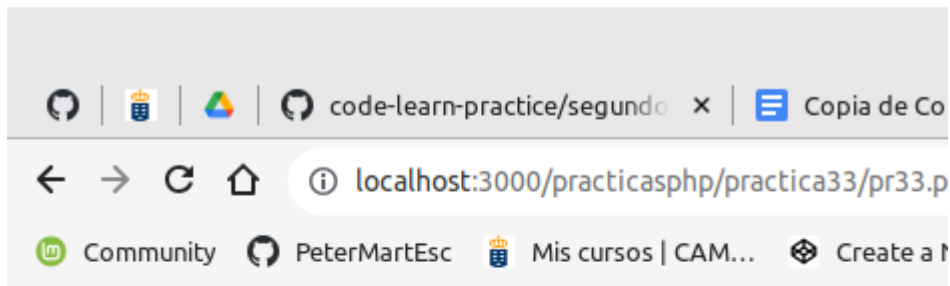
$recibido = $_GET["prueba"] ?? "nadita";
$recibido2 = $_GET["prueba2"] ?? "nadita";

// Mostramos los resultados
echo "<h3>Se ha recibido:</h3>";
echo "prueba: " . $recibido . "<br>";
echo "prueba2: " . $recibido2 . "<br>";
?>
```

En la primera captura podemos ver el php lanzado.



A continuación podemos ver lo que devuelve al usar el primer enlace, sin urlEncode. Como podemos ver devuelve la cadena entera.



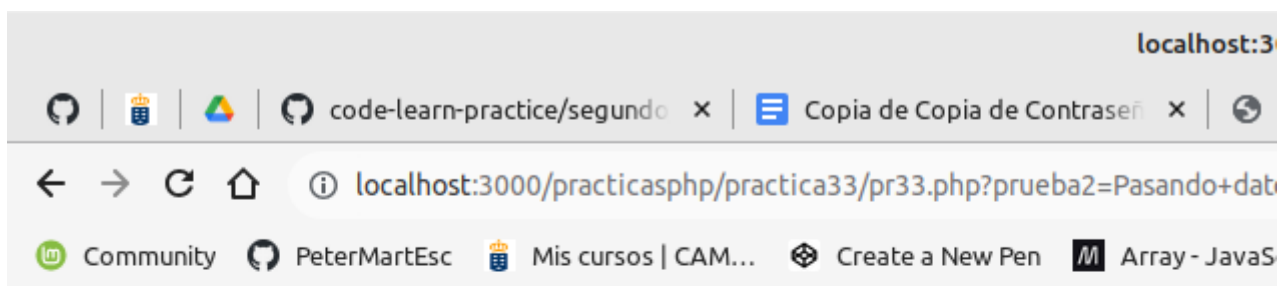
[Pasando datos sin urlEnco](#)

[Pasando datos con urlencode](#)

Se ha recibido:

prueba: Pasando datos diría.. que hay que usar urlencode
prueba2: nadita

Y en esta captura podemos ver lo que devuelve usando el urlencode. Aquí también devuelve la cadena entera.



[Pasando datos sin urlEnco](#)

[Pasando datos con urlencode](#)

Se ha recibido:

prueba: nadita
prueba2: Pasando datos diría.. que hay que usar urlencode

Practica 34

Recorrer el array \$_GET con un foreach y mostrar el conjunto de clave valor para la actividad anterior

```
<?php

$texto = "Pasando datos diría.. que hay que usar urlencode";

// Codificamos el texto con urlencode
$conUrlEncode = urlencode($texto);
```

```
// Creamos un enlace con los dos parámetros: prueba (sin urlencode) y
prueba2 (con urlencode)
echo "<a href='pr34.php?prueba={\$texto}'>Pasando datos sin urlEnco</a>";
echo "<br><br>";
echo "<a href='pr34.php?prueba2={\$conUrlEncode}'>Pasando datos con
urlencode</a>";

$recibido = $_GET["prueba"] ?? "nadita";
$recibido2 = $_GET["prueba2"] ?? "nadita";

// Mostramos los resultados
echo "<h3>Se ha recibido:</h3>";
echo "prueba: " . $recibido . "<br>";
echo "prueba2: " . $recibido2 . "<br>";

foreach ($_GET as $clave => $valor) {
    echo "Clave: $clave, Valor: $valor <br>";
}

?>
```

Practica 35

Realiza una página con un formulario que se llame a si misma para mostrar la tabla de un número introducido por el usuario. Se deberá controlar que el usuario haya introducido un número entero positivo. Hacer uso para ello de la función: `is_int()` buscando su funcionamiento en el manual oficial: [php.net](https://www.php.net)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="style.css">
    <title>Formulario Tabla</title>
</head>
<body>
    <form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
        <p>Introduzca el numero de la tabla: </p>
        <input type="number" name="numero">
        <input type="submit" name="enviarValor" value="enviar">
    </form>
</body>
</html>

<?php

if(!isset($_REQUEST["numero"])){
    exit();
}
```

```

$numero = intval($_REQUEST["numero"]);
$resultado = "";

if($numero < 0 || !is_int($numero)){    //No está haciendo mucho

    echo "<p id='error'>El valor introducido es negativo o no es un
numero</p>";

} else {
    echo "<table><tr><td>";
    for($i = 1 ; $i <=10 ; $i++){
        $resultado .= $numero . " * " . $i . " = " . ($numero*$i) . "<br>";
    }
    echo $resultado;
    echo "</td></tr></table>";
}
?>

```

Practica 36

Realizar una página con un formulario que se llame a si misma donde el usuario introduzca en un input una cadena de números separada por espacios (ej: 2 5 8 7 3 4) y muestre un número por línea, mostrando primero los números impares y luego los pares. (hacer uso de la función usort() y de la función explode())

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Separar por pares e impares</title>
</head>
<body>
<form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>"
    <p>Introduzca una cadena de numeros saprada por espacios: </p>
    <input type="text" name="numeros">
    <input type="submit" name="enviarValor" value="enviar">
</form>
</body>
</html>

<?php

if(!isset($_REQUEST["numeros"])){
    exit();
}

$numeros = $_REQUEST["numeros"];
$array = explode(" ", $numeros); //explode funciona como preg_split pero es
más limitada

function ordenarParImpar($a, $b){

```

```

    $a_par = $a % 2 === 0; //Se está guardando como booleano
    $b_par = $b % 2 === 0;

    // Comparar si uno es par y el otro no, compara los booleanos
    if ($a_par !== $b_par) {
        return $a_par <=> $b_par; //aqui está devolviendo el valor, impares
van antes que pares
    }

    return $a <=> $b;
}

usort($array, "ordenarParImpar");

foreach($array as $valor){

    echo "$valor <br>";
}

?>

```

Practica 37

Realizar una página como la anterior que se valide a si misma. Obligando que el correo sea válido, que el nombre no sea vacío al igual que el género. Si los datos están correctamente introducidos se mostrarán por debajo de "Datos ingresados" si no superan la validación se dirá los campos que no la superan con texto en rojo.

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Pr 37</title>
</head>
<body>
    <div class="cabecera">
        <h1>Validación del formulario</h1>
    </div>
    <div class="cuerpo">
        <form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>"
method="post">

            <?php
                if (isset($_POST['enviar']) && (empty($_POST['nombre']))) {
                    echo '<p style="color: red;">Debe introducir su nombre.

</p>';
                }
            ?>
        <label for="nombre">Nombre: </label>        <!--El label va

```

```
unido a el id del input-->
    <input type="text" id="nombre" name="nombre">
    <br><br>

    <?php
    if (isset($_POST['enviar']) && (empty($_POST['correo']))) {
        echo '<p style="color: red;">Debe introducir su correo
electrónico.</p>';
    }
    ?>
    <label for="correo">Correo: </label>
    <input type="text" id="correo" name="correo">
    <br><br>

    <?php
    if (isset($_POST['enviar']) && (empty($_POST['paginaWeb']))) {
        echo '<p style="color: red;">Debe introducir su
dirección de página web.</p>';
    }
    ?>
    <label for="paginaWeb">Página Web: </label>
    <input type="text" id="paginaWeb" name="paginaWeb">
    <br><br>

    <?php
    //if (isset($_POST['enviar']) && (empty($_POST['comentario'])))
{
    //      echo '<p style="color: red;">Debe introducir un
comentario.</p>';
    //}
    ?>
    <label for="comentario">Comentario: </label>
    <textarea name="comentario" id="comentario">
    </textarea>
    <br><br>

    <?php
    if (isset($_POST['enviar']) && (empty($_POST['genero']))) {
        echo '<p style="color: red;">Debe seleccionar su género.
</p>';
    }
    ?>
    <label for="genero">Genero</label>
    <input type="radio" id="genero-mujer" name="genero"
value="mujer">
    Mujer
    <input type="radio" id="genero-hombre" name="genero"
value="hombre">
    Hombre
    <input type="radio" id="genero-otro" name="genero"
value="otro">
    Otro
    <br><br>
```

```
        <input type="submit" id ="enviar" name="enviar" value="Enviar">
    </form>
</div>

<?php

    if (isset($_POST['enviar'])) {

        if (!empty($_POST['nombre'])&&(!empty($_POST['correo']))&&
(!empty($_POST['paginaWeb']))&&(!empty($_POST['genero']))) {

            $nombre = $_POST['nombre'];

            $correo = $_POST['correo'];

            $paginaWeb = $_POST['paginaWeb'];

            $comentario = $_POST['comentario'];

            $genero = $_POST['genero'];

            print "<br><br>";
            print "<h3>Datos ingresados correctamente.</h3><br>";
            print "Nombre: ".$nombre."<br><br>";
            print "Correo: ".$correo."<br><br>";
            print "Pagina Web: ".$paginaWeb."<br><br>";
            print "Comentario: ".$comentario."<br><br>";
            print "Genero: ".$genero."<br><br>";

        }

    }

?>
</body>
</html>
```